

Module 11 – Natural Language Processing (NLP)

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Data Science	M11-A1	3 Hours	Python 3

GENERAL INSTRUCTIONS

Attempt all sections. Write your code in Python 3 unless stated otherwise. For coding tasks, submit your .py source file along with a screenshot of your program output. Label every task clearly with its section and task number. All work must be your own.

A Section A — Concept Application

Answer all questions. Each question presents a real-world scenario followed by a concept-specific question.

SCENARIO

**S
1**

You are building an automated customer support chatbot for a food delivery platform that handles queries like 'Where is my order?', 'I want to cancel my order', and 'My food arrived cold and late'.

Question: Identify two NLP techniques the chatbot must use to correctly understand and route these customer messages. Explain why each technique is necessary and describe what would happen to chatbot accuracy if either technique were removed.

SCENARIO

**S
2**

You are processing 50,000 raw customer reviews scraped from a food delivery app. The reviews contain mixed letter cases, special characters, emojis, numbers, and common filler words such as 'the', 'is', and 'a'.

Question: Describe the text preprocessing pipeline you would apply to this dataset before feeding it to any NLP model. Justify the order in which you would apply each step, and explain specifically why skipping stopwords removal could hurt downstream model performance.

S
3

SCENARIO

You are building a food item search feature that must rank menu descriptions by relevance to a customer's typed query, for example 'spicy chicken without onion'.

Question: Compare Bag of Words and TF-IDF as text representation methods for this ranking task. Which would you choose and why? Identify one limitation that your chosen method still has even after it is applied.

S
4

SCENARIO

You are training a Naive Bayes classifier to automatically label incoming food delivery support tickets as 'Delivery Issue', 'Food Quality Issue', or 'App Issue' using a dataset of 10,000 historical labelled tickets.

Question: Explain the core probabilistic assumption that Naive Bayes makes about its input features. Describe a realistic food delivery example where this assumption is violated, and explain how that violation would affect the model's classification accuracy.

S
5

SCENARIO

You are comparing a Logistic Regression model and a Naive Bayes model for classifying food delivery reviews as Positive, Neutral, or Negative. Your dataset is heavily imbalanced: 80% of reviews are Positive, 15% Neutral, and 5% Negative.

Question: Describe two evaluation metrics you would prioritise over accuracy when comparing these models on this imbalanced dataset. Explain one technique you would apply to address the class imbalance before training.

S
6

SCENARIO

You are integrating a Hugging Face transformer pipeline into your food delivery app to automatically generate polite responses to customer reviews in real time, at a rate of several hundred requests per minute.

Question: Explain what the attention mechanism in transformers enables that traditional RNN-based models could not do effectively with long text sequences. Describe one practical challenge of running a transformer model in a high-traffic production environment and one common way to mitigate it.

B

Section B — Practical Coding Tasks

Complete all tasks. Each task targets one specific concept or feature from the module. Follow the requirements listed under each task.

Task 1: Food Review Text Preprocessor

Build a reusable text preprocessing function that cleans raw food delivery customer reviews for downstream NLP tasks.

- Accept a raw review string as input and return a cleaned, processed string.
- Convert text to lowercase, remove punctuation, and strip extra whitespace.
- Tokenise the cleaned text and remove English stopwords using NLTK's stopwords corpus.
- Apply stemming using PorterStemmer (or lemmatization using WordNetLemmatizer) to the token list and return the result as a single space-joined string. Test your function on at least three different food delivery review samples.

Task 2: TF-IDF Vectoriser for Menu Reviews

Apply TF-IDF vectorisation to a set of food delivery menu item reviews and identify the most informative terms per document.

- Create a list of at least 8 short food delivery review strings as your corpus (mix of positive, negative, and neutral tones).
- Use sklearn's TfidfVectorizer to transform the corpus into a TF-IDF matrix.
- Display the full TF-IDF matrix as a labelled pandas DataFrame with feature names as column headers and review index as row labels.
- For each review, print the top 3 words with the highest TF-IDF score and their scores.

Task 3: Complaint Category Classifier – Naive Bayes

Train a Multinomial Naive Bayes text classifier to automatically categorise food delivery support complaints into predefined issue types.

- Create a labelled dataset of at least 24 complaint strings across three classes: 'Delivery', 'Food Quality', and 'App'.
- Preprocess each complaint using the function from Task 1, then convert the corpus to a TF-IDF feature matrix.
- Split into 80% train and 20% test sets, train a MultinomialNB classifier, and generate predictions on the test set.
- Print the full classification report (precision, recall, F1-score per class) and the confusion matrix.

Task 4: Sentiment Analysis Pipeline – Model Comparison

Build a complete end-to-end NLP pipeline that preprocesses food delivery reviews, vectorises them, and compares Naive Bayes against Logistic Regression on a binary sentiment task.

- Create a labelled dataset of at least 30 food delivery reviews with 'Positive' and 'Negative' sentiment labels.
- Build two separate sklearn Pipeline objects – one combining TfidfVectorizer with MultinomialNB and one combining TfidfVectorizer with LogisticRegression.
- Train both pipelines and evaluate each on a held-out test set; collect accuracy, precision, recall, and F1-score.
- Print a side-by-side comparison table of both models' metrics and include a one-line comment in code stating which model you would deploy and your reason.

C

Section C – Mini Capstone Project

Mini Project: Food Delivery Review Intelligence System

Objective:

Build a console-based NLP application for a food delivery platform that lets staff add customer reviews, classifies each review by sentiment and complaint category, and generates a summary report – combining text preprocessing, TF-IDF vectorisation, and Naive Bayes or Logistic Regression classification into a single working program.

Your project must:

- The program must be menu-driven with at least 3 options: (1) Add a new review, (2) Classify a review and display the predicted sentiment and issue category, (3) View a summary report.
- Every review must pass through the full preprocessing pipeline (lowercase, punctuation removal, stopwords removal, stemming or lemmatization) before being stored or classified.
- The classifier must output both a sentiment label (Positive / Negative) and an issue category (Delivery / Food Quality / App / General) for every submitted review.
- The summary report must display: total reviews added, count per sentiment label, count per issue category, and the 5 most frequent content words across all stored reviews.
- The program must handle edge cases gracefully – if fewer than the minimum required training samples exist, it must display a clear message instead of crashing when classification is attempted.

D

Section D – AI-Augmented Learning

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a program that:

- Accepts a list of food delivery customer reviews as input (hardcoded or user-entered).
- Preprocesses each review using tokenisation, stopword removal, and stemming.
- Builds a TF-IDF matrix and trains a Logistic Regression model to classify reviews as Positive or Negative.
- Prints the model's accuracy on a test split and predicts the sentiment for three new reviews entered by the user at runtime.

STEP 2 • TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the code and find at least one bug, limitation, or improvement in the AI's solution. Fix it yourself.

SUBMIT

All 3 items required

- 1 The exact prompt(s) you gave the AI tool.
- 2 The AI's original code and your corrected version.
- 3 A 3–4 line note explaining what you changed and why the AI's version needed it.